

Relatório do Trabalho Prático

Programação Paralela com OpenMPI

Aron P. Sarabia, Cassiano C. De Souza

Faculdade de Computação (FACOM)
Universidade Federal do Mato Grosso do Sul (UFMS) – 79070-900
A Av. Costa e Silva, s/nº – Bairro Universitário
Campo Grande – MS – Brasil

{aron_petta, cassiano_c_souza}@ufms.br

Abstract. The present work focuses on presenting a resolution to the Jacobi Iterative Method through parallel programming via OpenMPI. Through performance analysis, we will verify the main points of reflection on the limits and challenges encountered in parallelizing the method's algorithm.

Resumo. O presente trabalho foca em apresentar uma resolução ao Método Iterativo de Jacobi através de programação paralela via OpenMPI. Através de análise de desempenho, verificaremos os principais pontos de reflexão sobre os limites e desafios encontrados na paralelização do algoritmo do método.

1. Introdução

Os sistemas lineares são utilizados na modelagem de problemas como simulações físicas, análise estrutural, circuitos elétricos e processamento de dados. Entretanto, à medida que o tamanho desses sistemas aumenta, o custo computacional para encontrar soluções cresce significativamente, tornando necessária a utilização de técnicas de computação distribuída e paralela para reduzir o tempo de processamento.

O Método de Jacobi resolve um Sistema Linear através de sucessivas iterações de aproximação à um vetor-solução escolhido previamente. Quando há convergência com a solução, o sistema é interrompido.

A implementação do paralelismo, e principalmente das técnicas de MPI, no algoritmo do método, incluem fragmentar os processos do laço de iteração, relacionando os mesmos à threads específicas para otimizar a resolução e explorar ambientes de memória distribuída, nos quais cada processo possui seu próprio espaço de memória e a comunicação ocorre através de troca de mensagens. Isso permite que diferentes cálculos sejam processados em paralelo, evitando atrasos e dependências que poderiam comprometer o desempenho do algoritmo. Em contraponto, a técnica promove maior velocidade e precisão na busca pela solução do problema.

Além disso, esta implementação complementa as versões desenvolvidas anteriormente utilizando OpenMP (TP1) e CUDA (TP2). Enquanto o OpenMP explora o paralelismo em arquiteturas de memória compartilhada por meio de threads, e o CUDA utiliza o processamento massivamente paralelo de GPUs, o MPI amplia o estudo do Método de Jacobi para ambientes distribuídos, permitindo comparar diferentes abordagens de paralelização e avaliar seus impactos em desempenho, escalabilidade e eficiência computacional.

2. Ambiente Experimental

- **Processador:** 11th Gen Intel® Core™ i5-1135G7.
- **Número de núcleos e threads:** 4 Núcleos e 8 Threads.
- **Memória RAM:** 24Gb
- **Sistema Operacional:** Fedora Linux 43 (Workstation Edition).
- **Compilador e Versão:** mpirun (Open MPI) 5.0.9.
- **Biblioteca MPI utilizada:** #include <mpi.h>.
- **Número Máximo de Processos Testados:** 8.

3. Implementação

Em primeira instância, antes da compilação do código, o usuário define o tamanho da matriz do sistema linear e a quantidade de processos MPI que participarão da execução. Em seguida, a função principal inicializa o ambiente MPI e chama um conjunto de funções responsáveis por executar o Método de Jacobi, considerando o número máximo de iterações e a tolerância de convergência previamente estabelecida. Todas as execuções são iniciadas e finalizadas por rotinas de medição de tempo, permitindo a análise de desempenho. Durante esse processo, uma função gera a matriz estritamente diagonal dominante e o vetor de termos independentes, outra executa a versão sequencial do algoritmo, uma terceira realiza a implementação paralela utilizando MPI e uma última organiza e imprime os resultados em formato de tabela, permitindo a comparação entre as abordagens.

A implementação sequencial inicia gerando um novo vetor de aproximação para a solução. Em seguida, executa um laço iterativo que permanece ativo enquanto o erro calculado estiver acima da tolerância e o limite máximo de iterações não tiver sido atingido. Durante cada iteração, a função percorre todas as linhas e colunas da matriz por meio de laços de repetição, realizando os cálculos das somas necessárias para obter os novos valores do vetor solução. Ao final de cada ciclo, o erro global é calculado para verificar a convergência do método.

Já na implementação paralela com MPI, a estratégia utilizada é a decomposição do domínio por linhas, distribuindo a matriz A e o vetor b entre os diferentes processos. Para isso, emprega-se o MPI_Scatterv, responsável por enviar a cada processo apenas o subconjunto de linhas que lhe pertence, utilizando os vetores sendcounts e displs para lidar corretamente com divisões não exatas da matriz. O vetor de aproximação inicial é distribuído para todos os processos por meio do MPI_Bcast, garantindo que cada um possua os dados necessários para os cálculos locais.

Durante cada iteração, cada processo calcula apenas os novos valores correspondentes às suas linhas locais, armazenando os resultados em um vetor próprio e acumulando um erro local. Como o Método de Jacobi exige que todos os elementos do vetor solução estejam disponíveis na próxima iteração, utiliza-se o MPI_Allgatherv para sincronizar os novos valores calculados entre todos os processos. Além disso, o MPI_Allreduce é empregado para somar os erros locais e determinar o erro global utilizado no critério de convergência. Dessa forma, todos os processos permanecem sincronizados e trabalham cooperativamente na resolução do problema.

A organização dos dados em memória é realizada através de estruturas lineares (*std::vector*), permitindo melhor localidade de acesso e simplificando o gerenciamento dos dados distribuídos. Cada processo enxerga apenas seu subconjunto de linhas da matriz, enquanto mantém uma cópia completa do vetor de aproximação necessário para os cálculos. Essa abordagem reduz a carga computacional individual e favorece a escalabilidade da aplicação.

Ao final da execução, os tempos obtidos e demais métricas de desempenho são organizados em uma tabela comparativa entre as versões sequencial e paralela. A análise dos resultados demonstrou que a implementação MPI proporcionou redução significativa no tempo de processamento para matrizes de maior dimensão, sendo o melhor desempenho observado com 4 processos, que apresentou um equilíbrio adequado entre o ganho computacional e o custo de comunicação. Também foi observado que o *speedup* cresce de forma mais evidente em problemas maiores, evidenciando a eficiência da decomposição por linhas e a viabilidade do MPI para aplicações de computação de alto desempenho..

3.1. Decisões de Projeto

A implementação do Método de Jacobi foi projetada para executar em ambiente distribuído utilizando MPI, permitindo que o número de processos seja definido pelo usuário durante a execução. Essa abordagem possibilita

dividir a carga computacional entre diferentes processos, reduzindo o tempo necessário para a resolução de sistemas lineares de grande porte.

A comunicação entre os processos foi realizada por meio de operações coletivas do MPI, como **MPI_Scatterv**, **MPI_Bcast**, **MPI_Allreduce** e **MPI_Allgatherv**, responsáveis pela distribuição dos dados, sincronização dos vetores de aproximação e cálculo do erro global. Paralelamente, a carga de trabalho foi distribuída por linhas da matriz, garantindo que cada processo fosse responsável por apenas uma parte do problema.

O critério de parada adotado considera a convergência do método, encerrando a execução quando o erro global torna-se inferior à tolerância especificada ou quando o número máximo de iterações é atingido. Os resultados demonstraram que essa estratégia apresentou boa escalabilidade, com destaque para a execução utilizando 4 processos, que ofereceu o melhor equilíbrio entre processamento e comunicação.

4. Resultados

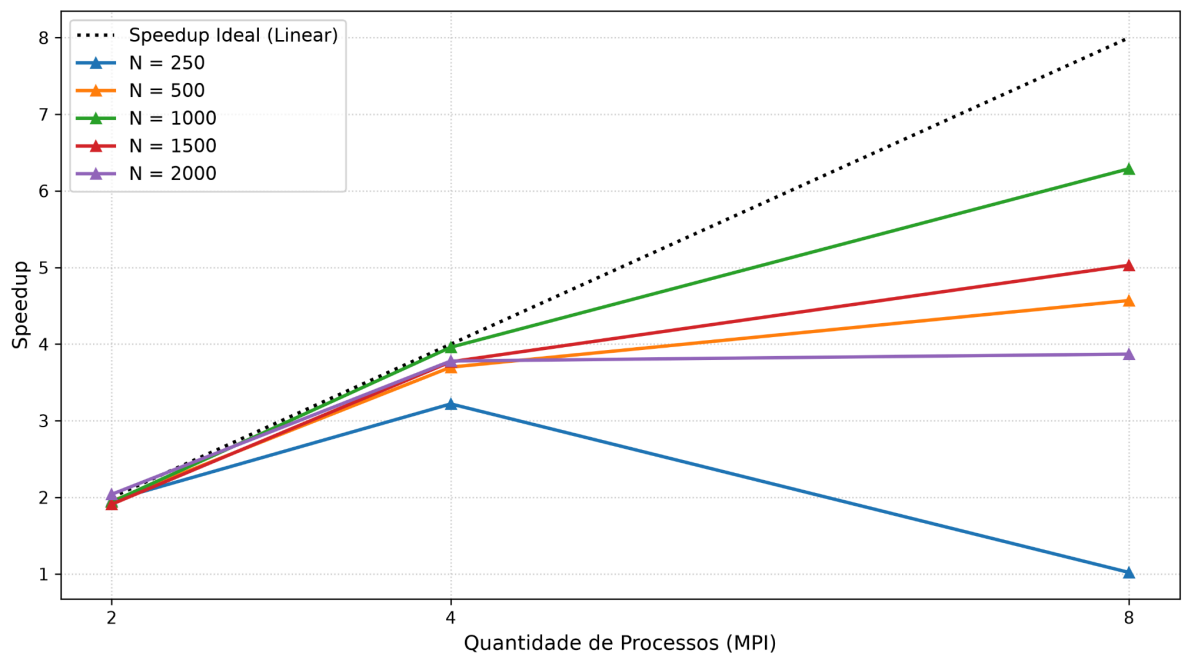
4.1. Desempenho

Tabela de comparação entre versões:

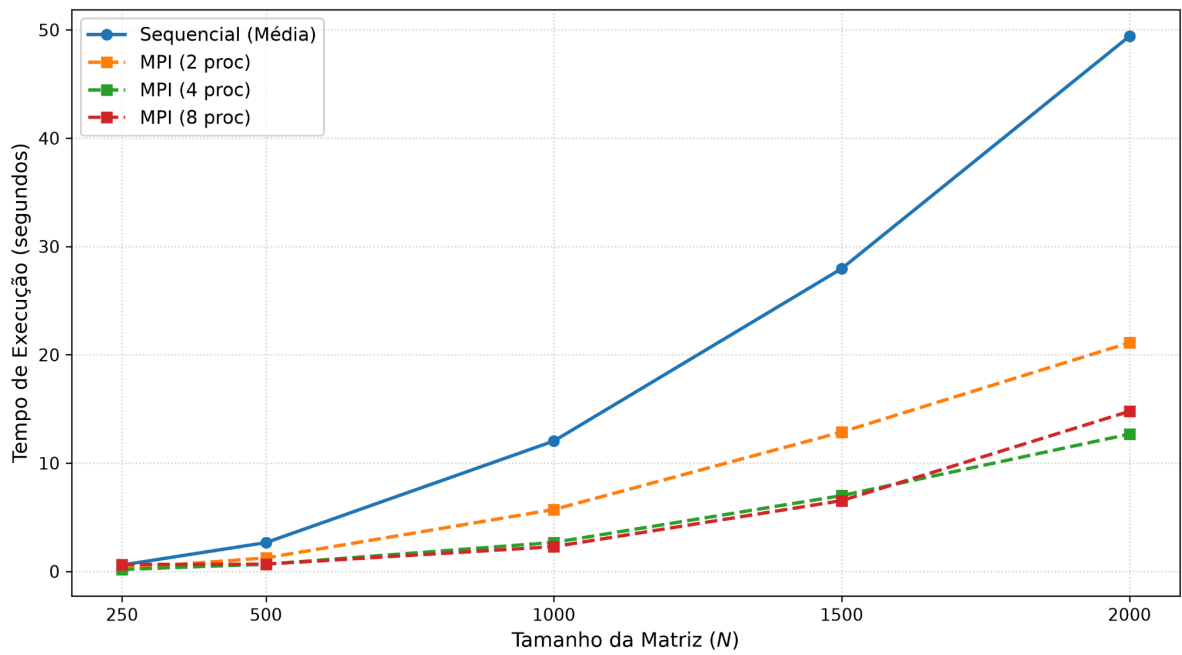
Versão	Configuração	Tempo(s)	Speedup vs Seq	Speedup vs OpenMP	Speedup vs CUDA
Sequencial CPU	1	28	1	1	1
OpenMP CPU (TP1)	4	47	3,75	-	-
CUDA GPU (TP2)	64	1,77	17,29	-	-
MPI (2)	2	13	6,5	3,6	0,13
MPI (4)	4	8	2	5,8	0,21
MPI (8)	8	7	1,4	6,7	0,24

Tabela 1: Comparação de desempenho entre implementação sequencial, OpenMP, CUDA e MPI.

Speedup obtido vs Quantidade de Processos



Tempo de Execução vs Tamanho da Matriz (N)



4.2. Convergência

Número de Iterações = 10000		
Número de Processos	Erro Sequencial	Erro MPI
2	14.025	14.025
2	98.813	98.813
2	331.507	331.507
2	484.946	484.946
2	579.180	579.180
4	11.853	11.853
4	105.482	105.482
4	324.918	324.918
4	474.938	474.938
4	559.075	559.075
8	11.383	11.383
8	103.758	103.758
8	334.821	334.821
8	474.237	474.237
8	585.014	585.014

Tabela 2: Convergência do método de Jacobi utilizando MPI

4.3. Discussão

A comparação entre MPI, OpenMP e CUDA evidencia diferentes formas de paralelização do Método de Jacobi. Enquanto o OpenMP utiliza *threads* em memória compartilhada e o CUDA explora o paralelismo massivo das GPUs, o MPI distribui o trabalho entre processos independentes, tornando-se mais adequado para ambientes de memória distribuída.

A influência do número de processos está diretamente relacionada ao equilíbrio entre processamento e comunicação. Embora mais processos reduzam a carga computacional individual, também aumentam a necessidade de sincronização dos dados. Nesse sentido, a configuração com 4 processos apresentou o melhor desempenho, oferecendo o melhor equilíbrio entre paralelismo e custo de comunicação.

Como pode ser observado nos resultados, o paralelismo apresenta maior destaque em matrizes maiores, onde a decomposição por linhas distribui uma quantidade significativa de trabalho para cada processo. A principal limitação da implementação está no custo das operações coletivas de comunicação, que podem se tornar um gargalo quando o número de

processos aumenta excessivamente, reduzindo os ganhos obtidos com a paralelização.

5. Conclusão

A implementação do Método de Jacobi com MPI demonstrou que a decomposição por linhas é uma estratégia eficiente para distribuir o processamento entre múltiplos processos. Os resultados evidenciaram ganhos significativos de desempenho, principalmente para matrizes de maiores dimensões, onde o paralelismo é mais bem aproveitado.

Os experimentos também mostraram que o aumento do número de processos nem sempre resulta em melhor desempenho, devido ao custo de comunicação e sincronização. Nesse contexto, a configuração com 4 processos apresentou os melhores resultados, consolidando o MPI como uma solução eficaz para aplicações de computação de alto desempenho.

6. Referências

 [MPI - Parallel and Distributed Computing Course: 7 Hours!](#)